

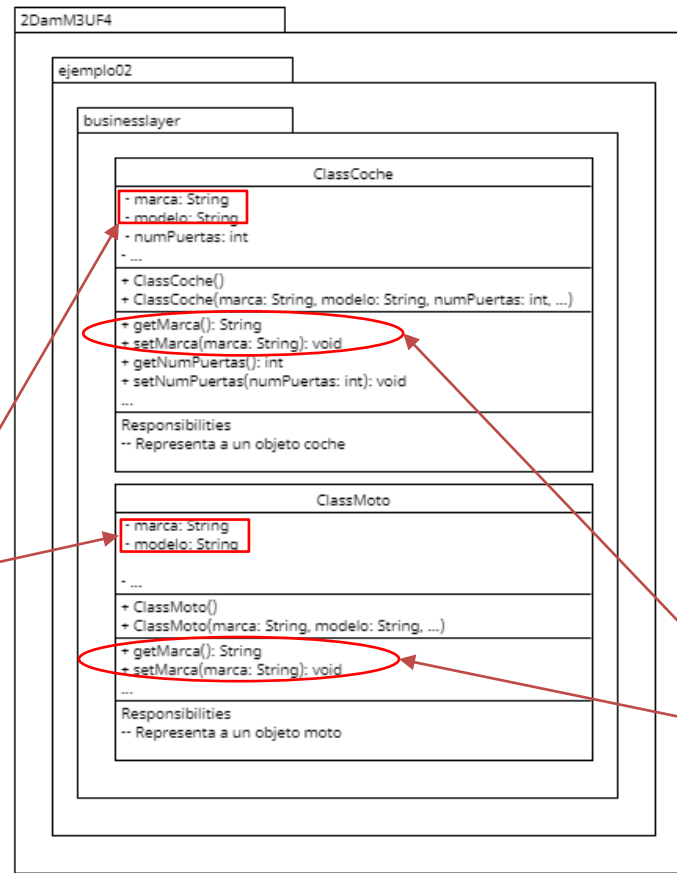
# $M_3-UF_4$ –HERENCIA y POLIMORFISMO

- Introducción
- Herencia
- Cómo se implementa en Java
- Polimorfismo
- Ejemplo

## Introducción

- La reutilización de código es un concepto crucial en la programación que evita la redundancia y minimiza los errores, facilitando la mantenibilidad y legibilidad del código.
- La Programación Orientada a Objetos (OOP) en Java ofrece mecanismos robustos para la reutilización de código, principalmente a través de la **herencia** y el **polimorfismo**.
- Vamos a introducir estos conceptos a través de un ejemplo:
  - Imagina que ya tenemos una clase Coche correctamente diseñada y ahora queremos una clase Moto. ¿Qué deberíamos hacer para implementar esta clase Moto?

## Herencia: Características



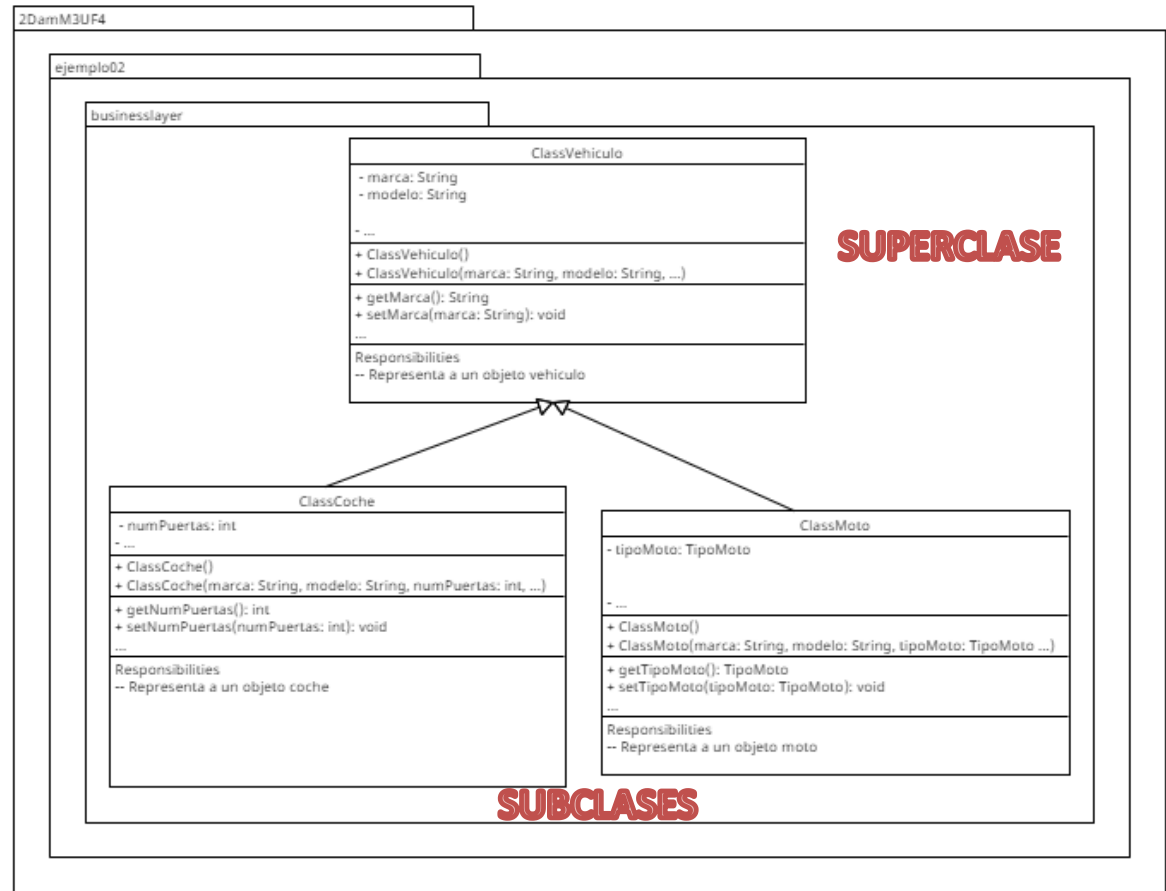
¿Cómo podemos reusar código?

Habr  una serie de atributos que ser n comunes!!  
No todos, algunos ser n espec ficos de cada clase

Habr  una serie de m todos que ser n comunes!!  
No todos, algunos ser n espec ficos de cada clase

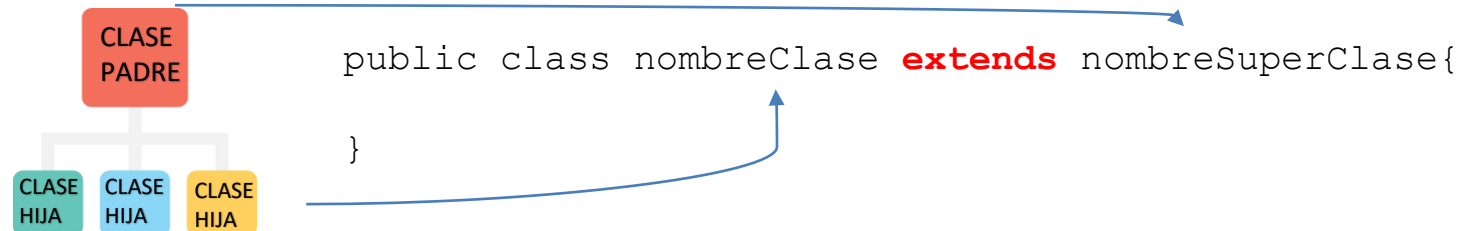
## Herencia:

Llevamos la parte común a la superclase (o clase padre) y dejamos la parte específica dentro de la clase correspondiente



## Herencia: Características

- La herencia permite que una clase (**la subclase**) **herede atributos y métodos de otra clase (la superclase)**. **Los constructores no se heredan**
- Esto promueve la **reutilización de código** y establece una relación natural entre la superclase (PADRE) y la subclase (HIJA) o, dicho de otro modo, al utilizar la herencia, puedes **escribir el código común en una superclase y luego “extender” esta superclase para añadir funcionalidades** específicas.
- **Una clase heredada sólo puede tener una superclase**
- Una subclase puede sobrescribir los métodos que hereda (@Override)
- Para indicar que una clase hereda de otra hay que usar la palabra reservada: **extends**



## Herencia

- ¿Cómo se implementa en Java?

### SUPERCLASE

Parte común

```
public class ClassVehiculo {  
    private String marca;  
    private String modelo;  
    private LocalDate fechaFabricacion;  
    private EnumTipoCombustible tipoCombustible;  
    private EnumColor color;  
    private double km;  
    private boolean parado;  
    private float velocidadActual;  
  
    // Constructores  
  
    public ClassVehiculo(){  
    }  
  
    public ClassVehiculo(String marca, String modelo, LocalDate fechaFabricacion,  
        EnumTipoCombustible tipoCombustible, EnumColor color, double km,  
        boolean parado, float velocidadActual) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.fechaFabricacion = fechaFabricacion;  
        this.tipoCombustible = tipoCombustible;  
        this.color = color;  
        this.km = km;  
        this.parado = parado;  
        this.velocidadActual = velocidadActual;  
    }  
}
```

## Herencia

- ¿Cómo se implementa en Java?

### SUBCLASE

```
package ejemplo01.businessLayer;

import java.time.LocalDate;
import java.time.Period;
import java.time.format.DateTimeFormatter;

public class ClassCoche extends ClassVehiculo{

    // 1.- Definir los atributos (PROPERTIES) (matricula, numMarchas, velocidadMaxima, autonomia, tra
    private int numPuertas;

    // 2.- Definir constructor sin parámetros, con todos los parámetros, con alguno de los parámetros
    public ClassCoche(){
        super();
    }

    public ClassCoche(String marca, String modelo, LocalDate fechaFabricacion,
        EnumTipoCombustible tipoCombustible, EnumColor color, double km,
        boolean parado, float velocidadActual, int numPuertas){
        super(marca, modelo, fechaFabricacion, tipoCombustible, color, km, parado, velocidadActual);
        this.numPuertas = numPuertas;
    }
}
```

Con `super()`  
invocamos al  
constructor SIN  
parámetros de la  
clase padre

Con `super(...)`  
invocamos al  
constructor CON  
parámetros de la  
clase padre



## Herencia

- ¿Cómo se implementa en Java?

### SUBCLASE

Con `super.toString`  
llamamos al  
método `toString`  
de la clase padre

```
public int getNumPuertas() {  
    return numPuertas;  
}  
  
public void setNumPuertas(int numPuertas) {  
    this.numPuertas = numPuertas;  
}  
  
@Override  
public String toString() {  
    StringBuilder sb = new StringBuilder();  
    sb.append(super.toString());  
    sb.append("ClassCoche{");  
    sb.append(", numPuertas=").append(numPuertas);  
    sb.append('}');  
    return sb.toString();  
}
```

## Polimorfismo

- Imaginemos ahora que queremos crear en nuestro programa principal un método que muestre el detalle de un coche, o el detalle de una moto.
- ¿Cómo lo harías?

```
public static void imprimirDetallesMoto(ClassMoto m){  
    System.out.println(m.toString());  
}  
  
public static void imprimirDetallesCoche(ClassCoche c){  
    System.out.println(c.toString());  
}
```

```
imprimirDetallesCoche(coche1);  
imprimirDetallesMoto(moto1);
```

## Polimorfismo

- Usando Polimorfismo puedes tener un solo método que maneje ambos tipos, ya que tanto Coche, como Moto son subclases de Vehículo:

```
public static void imprimirDetallesVehiculo(ClassVehiculo v) {  
    System.out.println(v.toString());  
}
```

```
imprimirDetallesVehiculo(cochel);  
imprimirDetallesVehiculo(motol);
```

## Polimorfismo

- El polimorfismo permite que un objeto sea tratado como una instancia de una superclase.
- Puedes escribir código que trabaje con objetos Vehículo, pero que en realidad estén trabajando con objetos Coche o Moto.